



计算机系统概论

基本知识

关于硬件



- **PC (程序计数器)**：指向当前执行的指令（执行完后更新为下一条要执行的指令）。在 x86-64 架构下称为 RIP。
- **寄存器**：处理器内部以名字访问的快速存储单元。
 - 整数寄存器可存储地址或整数数据。
- **条件码寄存器**：保存最近执行的算术或逻辑指令的状态信息。
- **内存**：以字节编码的连续存储空间。

指令集

指令集是软硬件的接口，包含：指令集组成；指令集架构（ISA）：实现软硬件解耦。

操作系统

作用：1.防止应用程序被失控的应用滥用。
2.提供简单一致的机制控制低级硬件设备。

其他

Amdahl's 定律

- 假设系统运行一个应用程序所需的时间为 T_{old} ，其中某部分占比为 a ，其性能提升比例为 k 。
- 优化后的总执行时间为：
$$T_{\text{new}} = T_{\text{old}} \left[(1 - a) + \frac{a}{k} \right]$$

- 加速比为: $S = \frac{T_{old}}{T_{new}} = \frac{1}{(1-a) + \frac{a}{k}}$

编译流程

1. `.c` - 编译 → `.s` (汇编代码)
2. `.s` - 汇编 → `.o` (目标文件)
3. `.o` - 链接 → 可执行程序

并发与并行

- **并发**: 同时具有多个活动的系统。
- **并行**: 利用并发来提高系统运行速度。

数据单位与位模式

- **bit**: 一个二进制位。
- **byte**: 8 位。
- **word**: 16 位 (1 word = 2 bytes, 为早期 x86 十六位系统的遗留)。
- **位模式**: 长度为 n 的位序列, 可表示 2^n 个数。

C 语言运算符

- **逻辑运算符** (`&&`, `||`): 具有短路效应, 返回 0 或 1。
- **按位运算符** (`&`, `|`, `^`, `~`, `<<`, `>>`): 返回运算结果的二进制数。
- **移位运算**:
 - **逻辑右移**: 高位填充 0 (适用于无符号数)。
 - **算术右移**: 高位填充符号位 (适用于有符号数, 但对于负数可能不向零取整)。

在 Java 中, `>>` 表示算术右移, `>>>` 表示逻辑右移。

关于数学

十进制转二进制

- **整数**:
 - 除二取余, 倒序排列。
 - 直到商为 1, 从该 1 开始往上取余数。

- **小数：**
 - 乘二取整。
 - 当小数部分为 1 后变为 0，直到小数尾为 0。

十六进制

- **表示方法：**符号位用 `0x` 表示，每位十六进制数表示 4 位二进制数。
- **转换关系：**
 - 十进制 10-15 对应十六进制 A-F。
 - **二进制转十六进制：**从右到左，每 4 位分组，转换为对应的十六进制数。
 - **十六进制转二进制：**反向操作。
- **有符号数：**若最高位在范围 8-F，表示负数（注意前导 0 会被忽略，记得看位数）。
- 取反是和为15而非16

数据容量

- $1\text{ KB} = 2^{10}\text{ B} = 1024\text{ 字节}$ 。
- 后续单位依次为： $1\text{ MB} = 1024\text{ KB}$ $1\text{ GB} = 1024\text{ MB}$ $1\text{ TB} = 1024\text{ GB}$ $1\text{ PB} = 1024\text{ TB}$

数据

整数运算

整数的运算本质上是模运算形式。

机器字

- **Machine Word：**表示一次整数运算处理的二进制位数。
- **地址：**机器字地址为其首字节地址；相邻地址间隔4 Byte（32位）或8 Byte（64位）。
- 指针的长度和地址的长度一致。
- **字节序：**
 - 小端：低地址存低位字节（如x86, RISC-V）。
 - 大端：低地址存高位字节（如PowerPC, Internet）。

- 小端表示变 大端表示例：0x12345678 → 0x78563412
`show_byte` 按地址从低到高打印字节，受字节序影响。

C语言中基本数据类型的大小 (单位为字节)

C Data Type	32-bit	x86-32	x86-64	
char	1	1	1	
short	2	2	2	
int	4	4	4	
long	4	4	8	(Linux)
long long	8	8	8	
float	4	4	4	
double	8	8	8	
long double	8	10/12	10/16	
char * or any other pointer	4	4	8	
	/	/	/	

(唯一——一个与机器取值范围相关的是long)

指int一般用esi

整数二进制编码

- **无符号数** (B2U) : $B2U(x) = \sum_{i=0}^{w-1} x_i \cdot 2^i$
- **带符号数** (补码 B2T) : $B2T(x) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$ (转人工: 绝对值取反码 + 1)

无符号数与带符号数转换：仅改变解读方式，二进制表示不变。

常数默认带符号（除非后加U）

操作时带符号数默认转为无符号数：如比较，计算等。故不能仅仅因为取值范围非负而使用无符号！

运算特点

1. **无符号加法**：溢出时结果为 $u + v - 2^w$ 。（放弃进位）
 $+^u_w$ (w指的是位长)
2. **补码加法溢出检测**：

$$TAdd_w(u, v) = \begin{cases} u + v + 2^w & u + v < TMin_w \\ u + v & TMin_w \leq u + v \leq TMax_w \\ u + v - 2^w & TMax_w < u + v \end{cases} \quad \begin{matrix} NegOver/向下溢出 \\ \\ PosOver/向上溢出 \end{matrix}$$

- 正溢出: $x > 0, y > 0, s \leq 0$
- 负溢出: $x < 0, y < 0, s \geq 0$

补码加法符号: $+^t_w$

3. **补码减法**: 减数取反加一后与被减数相加 (而不是取反减一啊啊啊)
实现了加法和减法在电路层面的统一

位移运算

- **无符号数右移**: $u \gg k = \lfloor u/2^k \rfloor$
- **带符号数右移**: 通过偏置量处理负数取整: $\lfloor (x + 2^k - 1)/2^k \rfloor$

浮点数

浮点数表示范围广但精度有限, 其数值: $V = (-1)^s \cdot M \cdot 2^E$

- **s**: 符号位
- **M**: 有效数 (位于[1.0, 2.0))。
- **E**: 指数 ($E = \text{exp} - \text{Bias}$, Bias为 $2^{n-1} - 1$)。

实际储存形式: s exp frac

规格化与非规格化浮点数

1. **规格化**: $\text{exp} \neq 00\dots 0$ 且 $\neq 11\dots 1$ 。
 - $E = 1 - \text{bias}$
 - 小数部分隐含1位, 范围[1.0, 2.0)。
2. **非规格化**: $\text{exp} = 00\dots 0$, 尾数范围小于1, 精度逐渐降低。

特殊值

- $\text{exp} = 11\dots 1, M = 00\dots 0$: 表示无穷或溢出值。
- $\text{exp} = 11\dots 1, M \neq 00\dots 0$: NaN。
- 浮点數位表示有限, 可表示的数值少于同位数定点数。

运算特性

- 加减法：对齐指数，操作尾数后规范化。
- 乘法：尾数相乘，指数相加，规范化结果。
- 舍入：采用向偶数舍入策略，避免统计偏差。

《如何获得浮点数》

一个例子：

▶ $15213_{10} = 11101101101101_2 = 1.1101101101101_2 \times 2^{13}$

▶ 尾数

$M = 1.1101101101101_2$
 $\text{frac} = 11011011011010000000000_2$

▶ 阶码

$E = 13$
 $\text{Bias} = 127 \quad (2^{e-1}-1) \quad e=8$
 $\text{Exp} = 140 = 10001100_2$

Hex:	4	6	6	D	B	4	0	0
Binary:	0100	0110	0110	1101	1011	0100	0000	0000
140:	100 0110 0							
15213:	1110 1101 1011 01							

向偶数舍入：四舍五入；逢5，若为偶数则舍去，若为奇数则进位
(奇偶出现概率相当，有进有退，在统计意义上持平)

数值	二进制表示	舍入后	具体的舍入操作	最终结果
2 3/32	10.00011_2	10.00_2	<1/2—down	2
2 3/16	10.00110_2	10.01_2	>1/2—up	2 1/4
2 7/8	10.11100_2	11.00_2	1/2—up	3
2 5/8	10.10100_2	10.10_2	1/2—down	2 1/2

类型转换

- 浮点数转整型：截断尾数，溢出或NaN结果未定义（常设为 T_{min} 或 T_{max} ）。
- 整型转浮点数：精确转换为double；转换为float可能舍入但不溢出。

汇编语言语法

需要掌握的要点：程序语义的等价转换，指令/函数/数据的定位

相关知识补充

内存与寄存器的比较

- 1. **内存**：大容量，慢速。地址表示，多种方式形成
- 2. **寄存器**：小容量，快速。用名字表示

通用寄存器

- 一般来说，寄存器以 `%r` 开头表示64位，`%e` 表示32位，无前缀表示16位
- 在x86-64中，除了 `%rsp`（栈指针寄存器，指明运行时栈的结束位置）外，其他寄存器都可视作通用寄存器

32位寄存器的分解表示

32位寄存器	16位寄存器	8位寄存器（高位）	8位寄存器（低位）
<code>%eax</code>	<code>%ax</code>	<code>%ah</code>	<code>%al</code>
<code>%ebx</code>	<code>%bx</code>	<code>%bh</code>	<code>%bl</code>
<code>%ecx</code>	<code>%cx</code>	<code>%ch</code>	<code>%cl</code>
<code>%edx</code>	<code>%dx</code>	<code>%dh</code>	<code>%dl</code>

参数传递

- 当参数少于7个时，参数从左到右放入寄存器：`rdi, rsi, rdx, rcx, r8, r9`（注意顺序）
- 当参数为7个及以上时，前6个仍然传入寄存器，后续的参数则从右向左放入栈中

地址和内存操作

- 用 `$r_a$` 表示任意寄存器，`R[$r_a$]` 表示其值
- 用 `$M_b[Addr]$` 表示从地址 `Addr` 开始的 `b` 个字节的内存引用

从C代码生成汇编代码

- 生成汇编代码的命令：

```
gcc -Og -S sum.c # -O是优化选项，可能会使生成的汇编代码与C代码不完全对应，-Og较为友好但优化
```

- 反汇编程序： `objdump -d sum` （从可执行文件或目标文件反向生成与机器码对应的汇编代码）
- 禁用位置无关代码生成： `-fno-PIC`
- 禁用堆栈保护机制： `-fno-stack-protector` （防止堆栈溢出攻击）

x86-64指令特点

- 支持多种类型的指令操作数
- 算术与逻辑指令可以以内存数据为操作数
- 支持多种内存地址计算模式
- 变长指令

汇编语言数据格式

1. **数据类型**：整型（表示数值或内存地址）；浮点数。无复杂数据类型
2. **后缀**： `B`，`W`，`L`，`Q` 对应1, 2, 4, 8字节

地址表达式四要素

- `D(Rb,Ri,S)`：表示内存地址，格式为 `Mem$[R[R_b]+S*R[R_i]+D]$`
 - `D`：常量（地址偏移量）
 - `S`：比例因子（1, 2, 4, 或 8）
 - `Rb`：基址寄存器（16个通用寄存器之一）
 - `Ri`：索引寄存器（`%rsp`不作为索引寄存器）
 - 可选：如果缺省则视为0

地址表达式	地址计算	结果
<code>0x8(%rdx)</code>	<code>0xf000 + 0x8</code>	<code>0xf008</code>
<code>(%rdx,%rcx)</code>	<code>0xf000 + 0x100</code>	<code>0xf100</code>
<code>(%rdx,%rcx,4)</code>	<code>0xf000 + 4*0x100</code>	<code>0xf400</code>
<code>0x80(%rdx,2)</code>	<code>2*0xf000 + 0x80</code>	<code>0x1e080</code>

数据转送指令 `mov`

- `mov Source, Dest`

- 支持操作数类型：
 - 立即数：例 `$0x400`（注意前缀 `$`）
 - 寄存器：16个通用寄存器之一
 - 内存：8个连续字节，起始地址由地址表达式指定
- 示例：

源操作数		目的操作数	源（左侧），目的（右侧）	类似的C语言表示
movl	Imm	Reg	<code>movq \$0x4,%rax</code>	<code>temp = 0x4;</code>
		Mem	<code>movq \$-147,(%rax)</code>	<code>*p = -147;</code>
	Reg	Reg	<code>movq %rax,%rdx</code>	<code>temp2 = temp1;</code>
		Mem	<code>movq %rax,(%rdx)</code>	<code>*p = temp;</code>
	Mem	Reg	<code>movq (%rax),%rdx</code>	<code>temp = *p;</code>

- 注意：
 - `%rax` 是寄存器地址，`(%rax)` 是内存中的地址
- `movzbl`：零扩展，`movsbl`：符号扩展

只有32位到64位会零扩展，低8/16位到64位不会

零扩展有时会导致符号错误（例：-1（11111111）变成255（0000000011111111））、溢出等问题

地址计算指令 `leal`

- `leal Source, Dest`：计算地址并将其赋值给 `Dest`（寄存器）
 - 示例：
 - 地址计算（不访存）
 - 进行如 `x + k * y` 这一类型的整数计算，其中 `k = 1, 2, 4, 8`
 - 注意**：返回的是地址而非值，返回地址的位数和操作系统位数一致
 - `leal 0xffffffff(%eax), %ecx` 相当于将 `%eax - 1` 赋给 `%ecx`

整数计算指令

双操作数指令	指令执行的操作	单操作数指令	指令执行的操作
addq Src, Dest	Dest = Dest + Src	incq Dest	Dest = Dest + 1
subq Src, Dest	Dest = Dest - Src	decq Dest	Dest = Dest - 1
imulq Src, Dest	Dest = Dest * Src	negq Dest	Dest = -Dest
salq Src, Dest	Dest = Dest << Src	notq Dest	Dest = ~Dest
sarq Src, Dest	Dest = Dest >> Src		
shrq Src, Dest	Dest = Dest >> Src		
xorq Src, Dest	Dest = Dest ^ Src		
andq Src, Dest	Dest = Dest & Src		
orq Src, Dest	Dest = Dest Src		

- 地址表达式本身不访问内存。使用 `leaq` 计算时，不会访问其值，而是将其操作给 `Dest`。
注：Multiply（取低64位）与 `shll` 等价，算术右移与逻辑右移在 `sarq` 和 `shrq` 中分别表示

程序控制流与相关指令

- 当前执行程序的信息：临时数据（通用寄存器），栈顶地址（`%rsp`），当前指令地址（下一条指令 `%rip`），条件码

条件码

- **CF**：进/借位标志（有进位则置一）
- **SF**：符号标志（计算结果小于零则置一）
- **ZF**：零标志（为零置一）
- **OF**：溢出标志（补码运算溢出置一）

这些条件码由算术指令（不含 `leaq` 指令）隐式设置。

比较指令

- 改变条件码而不改变操作数
- `cmpq b, a`：类似于计算 `a - b`（右减左）
- `testq Src2, Src1`：计算 `Src1 & Src2`

读取条件码的指令 `SetX`

- 读取当前条件码，并存入目的字节寄存器。

SetX	涉及的条件码（组合）	语义
<code>sete</code>	ZF	Equal / Zero
<code>setne</code>	$\sim$ ZF	Not Equal / Not Zero
<code>sets</code>	SF	Negative
<code>setns</code>	$\sim$ SF	Nonnegative
<code>setg</code>	$\sim$ (SF^OF)& $\sim$ ZF	Greater (Signed)
<code>setge</code>	$\sim$ (SF^OF)	Greater or Equal (Signed)
<code>setl</code>	(SF^OF)	Less (Signed)
<code>setle</code>	(SF^OF) ZF	Less or Equal (Signed)
<code>seta</code>	$\sim$ CF& $\sim$ ZF	Above (unsigned)
<code>setb</code>	CF	Below (unsigned)

例：

<pre>int gt(long x, long y) { return x > y; }</pre>	寄存器		用途
	<code>%rdi</code>		
	<code>%rsi</code>		
	<code>%rax</code>		
<code>cmpq %rsi, %rdi</code>	<code>%rdi</code>	# Compare x:y	“64-bit operands generate a 64-bit result in the destination general-purpose register. 32-bit operands generate a 32-bit result, zero-extended to a 64-bit result in the destination general-purpose register.” 摘自“Intel® 64 and IA-32 Architectures Software Developer’s Manual Volume 1: Basic Architecture”
<code>setg %al</code>	<code>%al</code>	# Set when >	
<code>movzbl %al, %eax</code>	<code>%eax</code>	# Zero rest of %rax	
<code>ret</code>			

条件跳转指令（jX 指令）

依赖当前的条件码选择下一条执行指令。

jX	涉及的条件码（组合）	语义
jmp	1	Unconditional
je	ZF	Equal / Zero
jne	~ZF	Not Equal / Not Zero
js	SF	Negative
jns	~SF	Nonnegative
jg	~(SF^OF)&~ZF	Greater (Signed)
jge	~(SF^OF)	Greater or Equal (Signed)
jl	(SF^OF)	Less (Signed)
jle	(SF^OF) ZF	Less or Equal (Signed)
ja	~CF&~ZF	Above (unsigned)
jb	CF	Below (unsigned)

直接跳转：目标地址在编译阶段已经确定，比如函数是同一个模块的静态函数（仅在本模块内部可见）。编译器在生成目标代码时，可以直接将这些地址硬编码到指令中。

间接跳转：目标地址在编译阶段无法确定，可能依赖于运行时的计算或在程序执行时才能确定。例如，通过函数指针调用的函数或跨模块的函数调用。间接跳转通常是通过某种指针来实现的，因此需要在运行时确定目标地址。

对于**jmp**，带有*标志的jmp指令表示这是一个间接跳转。这种跳转的目标地址是由某个寄存器或内存位置存储的，而不是一个直接的硬编码地址。例如，通过函数指针或表格实现的跳转；不带*的jmp指令通常是直接跳转，即跳转目标在编译阶段已经可以确定

条件移动指令

语义： `if (Test) Dest <- Src`

- 用于避免条件跳转指令的负面影响，尤其是在现代流水线处理器中。
- 示例： `cmove %rbx, %rax`：条件码ZF置1时，将%rbx的值传给%rax
- 实质：宁愿都算（猜测执行），也不做条件跳转，除非猜测代价较高（比如Then-Expr 或Else-Expr 表达式有“副作用”（比如更改一个变量的值等）或计算量较大）。

考试例题

例题

```
#include <stdlib.h>

#define likely(x) __builtin_expect(!!(x), 1)
#define unlikely(x) __builtin_expect(!!(x), 0)
int main(int argc, char *argv[])
{
    int a,b;
    /* Get the value from somewhere GCC can't optimize */
    a = atoi(argv[1]); /* string to int */
    b = a*a;
    if (unlikely(a == 2))
    // if (likely(a == 2))
    {
        a++; b++;
    }
    else
    {
        a--; b--;
    }
    return a+b;
}
```

» main:

```
subq $8,%rsp
movq 8(%rdi),%rdi
xorl %esi,%esi
movl $10,%edx
call strtol          #atoi调用,
                      #返回值在eax中
movl %eax,%esi      #esi = a
movl $3,%ecx        #ecx = 3
imull %eax,%esi     #esi (b) = a^2
cmpl $2,%eax
leal 1(%rsi),%edx    #edx = b++
je .L3
leal -1(%rax),%ecx   #ecx = a--
leal -1(%rsi),%edx   #edx = b--
```

» .L3:

```
leal (%rcx,%rdx),%eax
addq $8,%rsp
ret
```

gcc -O2

例题

```
» main:
subq $8,%rsp
movq 8(%rdi),%rdi
xorl %esi,%esi
movl $10,%edx
call strtol
movl %eax,%ecx
imull %eax,%ecx
cmpl $2,%eax
jne .L2
leal 1(%rcx),%eax
movl $3,%edx
```

» .L3:

```
addl %edx,%eax
addq $8,%rsp
ret
```

» .L2:

```
leal -1(%rax),%edx
leal -1(%rcx),%eax
jmp .L3
```

问题

分析编译指示 (directives)

```
#define likely(x) __builtin_expect(!!(x), 1)
#define unlikely(x) __builtin_expect(!!(x), 0)
```

的作用——为何生成的指令序列的顺序不同，与处理器流水线的运行过程有何关系？

思想：通过调整可能性把可能性较小的写到跳转（从而减少时间损耗）

循环

Do-While循环

```

movl $0, %eax      # result = 0
.L2:               # loop:
    movq %rdi, %rdx
    andl $1, %edx   # t = x & 0x1
    addq %rdx, %rax # result += t
    shrq %rdi      # x >>= 1
    jne .L2        # if (x) goto loop
ret

```

While:

```

goto test;
loop:
    Body
test:
    if (Test)
        goto loop
done;

```

For循环

```

Init;
while(Test){
    Body;
    Update;
}

```

switch

- 比较跳转：通常在case数值较少或分布较稀疏时有效
- 使用跳转表

```

void switch_eg(long x, long n,
               long *dest)
{
    long val = x;

    switch (n) {

    case 100:
        val *= 13;
        break;

    case 102:
        val += 10;
        /* Fall through */

    case 103:
        val += 11;
        break;

    case 104:
    case 106:
        val *= val;
        break;

    default:
        val = 0;
    }

    *dest = val;
}

```

```

void switch_eg(long x, long n, long *dest)
x in %rdi, n in %rsi, dest in %rdx
1  switch_eg:
2      subq    $100, %rsi           Compute index = n-100
3      cmpq    $6, %rsi            Compare index:6
4      ja     .L8                  If >, goto loc_def
5      jmp     *.L4(,%rsi,8)        Goto *jt[index]
6  .L3:                                loc_A:
7      leaq    (%rdi,%rdi,2), %rax   3*x
8      leaq    (%rdi,%rax,4), %rdi   val = 13*x
9      jmp     .L2                  Goto done
10 .L5:                                loc_B:
11      addq    $10, %rdi            x = x + 10
12 .L6:                                loc_C:
13      addq    $11, %rdi            val = x + 11
14      jmp     .L2                  Goto done
15 .L7:                                loc_D:
16      imulq   %rdi, %rdi           val = x * x
17      jmp     .L2                  Goto done
18 .L8:                                loc_def:
19      movl    $0, %edi             val = 0
20 .L2:                                done:
21      movq    %rdi, (%rdx)         *dest = val
22      ret                          Return

```

栈操作指令

栈是一块特定的内存区域，用于存储函数调用时的数据，包括函数参数、局部变量、返回地址等。栈操作指令用于控制栈的读取和写入。

pushq 指令

- 语法: `pushq Src`
- 执行: `%rsp = %rsp - 8; Mem[%rsp] = Src`
- 功能: 将 `Src` 压入栈中, 并更新栈指针 `%rsp`。

popq 指令

- 语法: `popq Dest`
 - 执行: `Dest = Mem[%rsp]; %rsp = %rsp + 8`
 - 功能: 从栈中弹出一个值并存入 `Dest`, 更新栈指针 `%rsp`。
-

函数调用与返回

在程序中, 函数调用会涉及到栈的操作, 特别是返回地址的存储以及栈帧的创建。

call 指令

- 语法: `call label`
- 执行: `%rip = label; Mem[%rsp] = %rip - 4`
- 功能: `call` 指令将当前的指令地址 (即返回地址) 压入栈中, 然后跳转到 `label` 所在位置开始执行。

ret 指令

- 语法: `ret`
 - 执行: `%rsp = %rsp + 8; %rip = Mem[%rsp]`
 - 功能: `ret` 指令弹出栈中的返回地址, 并跳转到该地址处继续执行, 恢复 `%rsp` 的值。
-

数组

一维数组

连续存储, 每个元素占据相同大小的空间 (取决于数据类型)

$[BASE + i * TYPE_SIZE]$

嵌套数组

行优先存储

$[BASE + (i * n + j) * TYPE_SIZE]$

multi-level array

指针数组，每个指针指向另一个数组

`mov [arr + i * PTR_SIZE], eax`

`mov [eax + j * TYPE_SIZE], eax`

(看似连续存储，实际是离散的)

二维数组/多维数组

$[BASE + (i * n + j) * TYPE_SIZE]$

$[BASE + ((i * y + j) * z + k) * TYPE_SIZE]$ (三维)

.....

动态数组

例子：

```
int var_ele(size_t n, int a[n][n], size_t i, size_t j) { return a[i][j];}
n in %rdi, a in %rsi, i in %rdx, j in %rcx
imulq %rdx,%rdi
leaq (%rsi, %rdi, 4), %rax
movl (%rax, %rcx, 4), %eax
ret
```

数组的连续访问可以优化：改为每次加定值而非一直进行重复计算

结构与内存对齐

1. 结构的存储

- 存储方式
 - 结构体在内存中存储为**连续分配的内存区域**，各个域按照声明顺序存储，访问时通过名字访问。
 - 结构体的整体大小和各域的位置由编译器决定，可能会进行内存对齐优化以提高访问效率。
- 内存对齐的原因
 - **形式上理解**：确保所有数据不会跨块访问。
 - **实际原因**：计算机内存按块访问，对齐可以防止跨块读取，提高效率。
- x86-64 下对齐规则
 - 不同数据类型的对齐要求：
 - 1 byte (e.g., `char`): 无需对齐。
 - 2 bytes (e.g., `short`): 地址最后一位为 0。
 - 4 bytes (e.g., `int`, `float`): 地址最后两位为 0。
 - 8 bytes (e.g., `double`, `long`, `long long`, `pointer`): 地址最后三位为 0。
 - 16 bytes (e.g., `long double`): 地址最后四位为 0。
 - 结构的对齐要求
 - 结构体的对齐要求等于其所有成员中对齐要求的最高值。
 - 如果结构体嵌套，取所有嵌套结构中对齐要求的最高值。

2. 联合体

- 联合体的所有成员共享同一块内存区域，其大小由最大成员的大小决定。
- 具体使用的是哪个需要按照函数来决定/变量特征来猜测

结构的传入与传出

1. 结构作为函数返回值

- **方法**：将结构体的地址存入 `%rax`，调用者通过 `%rax` 获得返回值地址。

2. 结构作为函数参数

- 方法：将结构体地址存入 `%rdi`，被调用函数通过 `%rdi` 访问参数地址。

示例说明

C99的VLA (variable-length array) 示例

```
extern long read_val();

long read_and_process(int n)
{
    long vals[n];

    for (int i = 0; i < n; i++)
        vals[i] = read_val();
    return vals[n-1];
}
```

```
read_and_process:
    pushq   %rbp
    movq    %rsp, %rbp

    pushq   %r14
    pushq   %r13
    pushq   %r12
    pushq   %rbx

    movl     %edi, %r13d      # %r13d = n
    movslq   %edi, %rax      # %rax = n
    leaq     22(,%rax,8), %rax # 8*n + 22 //long 类型
    andq     $-16, %rax      # 16字节对齐
    subq     %rax, %rsp      # 栈上分配空间
    movq     %rsp, %r14      # VLA地址?

    testl    %edi, %edi
    jle     .L2              # n<=0?
```

-O1 -fno-stack-protector -fno-PIC

42

0. 上面movl下面movq是因为下面一次挪了两个

1. 为何不直接使用现有地址，而用新地址？

- 这与C函数的调用规范有关。在调用时，struct参数一定是在栈上传递，而且起始地址是符合顺序递增的，如果是第一个栈上的参数，那它一定得在 $8(\%rsp)$ 的 ($\%rsp$) 是返回地址)

从上帝视角看确实如果让 `input_struct` 直接去用已经放好的这个 $24(\%rsp)$ 位置的 struct 会更好（在 `call` 之后应该是 $32(\%rsp)$ ）。但是从编译器的角度，没做编译优化的情况下，它就完全按规范来的，按规范去把这个struct 挪到 $8(\%rsp)$ 的位置（`call` 后），这就是红色汇编做的事。

2. 这里的-16代表16字节对齐？

$[-16]_{10} = [11111...1110000]_2$

将它与`%rax`做一个按位and 就使后面4位是0，即一定被16整除了

过程/函数

控制转移

详见汇编语言语法“函数调用与返回”

数据传递

传参

- 寄存器传递：前六个参数通过寄存器传递，具体为：
 - `%rdi` , `%rsi` , `%rdx` , `%rcx` , `%r8` , `%r9` , 返回值通过 `%rax` 返回。
 - 栈传递：超过六个参数通过栈传递，参数从右往左压栈。
-

栈帧

栈帧用于存储函数调用过程中的局部变量、返回地址、临时空间等数据。当进入函数时，程序会为该函数分配栈帧，函数返回时会释放栈帧。

栈帧的组成：局部变量，返回地址，临时空间，函数参数

栈帧指针

- `%rbp` (Frame Pointer) 指向栈帧的起始地址。
- `%rsp` (Stack Pointer) 指向栈顶地址，栈是从高地址向低地址增长。

在现代计算机中，由于编译器知道栈帧的大小，`%rbp`常常不再被保留，栈帧的管理通常通过`%rsp`进行。

栈帧的分配与释放

1. 栈帧分配：

- 栈帧的分配通过调整栈指针（`%rsp`）来实现。栈指针向下移动以腾出足够的空间来存储栈帧中的内容。
- 栈帧大小的计算依赖于函数的局部变量、参数和其他数据，编译器会在生成目标文件时分析栈的需求并分配足够的空间。

2. 栈帧释放：

- 栈帧的释放非常简单，只需要将栈指针（%rsp）加回栈帧的大小即可。
- 释放栈帧时，%rsp会恢复到函数调用之前的状态，栈中的数据也随之被清理。

3. 栈的对齐通常为16字节

一定入栈的情况

1. 要求地址的变量（哪怕是临时变量）
2. 寄存器不足
3. volatile变量(直接存取原始内存地址)

栈帧的操作特性：x86-64架构

1. **分配整个栈帧**：当函数被调用时，栈帧一次性分配，%rsp会减去栈帧的大小。
2. **栈内容访问**：栈帧中的数据通过%rsp来访问。%rsp指向栈顶，每次通过偏移量访问栈帧中的局部变量、参数等。
3. **延迟分配**：栈帧的分配可以延迟，最多可以直接访问栈上不超过128字节的空间。
4. **释放栈帧**：栈帧的释放是通过调整%rsp来实现的，栈指针会恢复到调用前的位置，完成栈空间的回收。

栈帧操作示例

假设我们有以下代码：

```
int func(int a, int b) {  
    int x = a + b;  
    return x;  
}
```

1. 栈帧的分配：

在 32 位机器上，栈帧的内容可以分解如下（从高地址到低地址）：

1. **返回地址**：调用 `func` 的下一条指令地址。
2. **参数 a 和 b**：由调用者压栈。（先b后a）

3. **局部变量 x**：在栈中分配的空间，用于存储函数内的局部变量。
4. **被保存的寄存器**：若函数使用了某些需要保存的寄存器（如 `%ebp`），它们的原值会被保存到栈中。
(从而反推压栈顺序)

2. 栈帧的释放：

- 当 `func` 执行完毕，程序会执行 `ret` 指令，将返回地址从栈中弹出，并跳转到调用 `func` 的地方。
- 栈指针 (`%rsp`) 会加上栈帧的大小，从而恢复到函数调用之前的状态，释放栈空间。

动画: "<https://sites.google.com/site/illustratedcomputer/stackframe>"

3. Memory Management (内存管理)

调用寄存器作为程序的临时存储

通用寄存器根据 ABI 的规定分为两类：

- **调用者保存 (Caller Saved)**：在调用子函数时，需要将这些寄存器的值保存在栈中，调用后再恢复。
- **被调用者保存 (Callee Saved)**：被调用的函数在使用这些寄存器之前必须将它们 的值保存在栈帧内，退出时恢复。

<code>%rax</code>	返回值-调用者保存	<code>%rbx</code>	被调用者保存
<code>%rcx</code>	第四个参数-第n个参数		倾向于调用者保存
<code>%rdx</code>	第三个参数	<code>%rsi</code>	第二个参数
<code>%rdi</code>	第一个参数	<code>%rsp</code>	栈顶指针
<code>%rbp</code>	被调用者保存	<code>%r8</code>	第五个参数
<code>%r9</code>	第六个参数	<code>%r10</code>	调用者保存
<code>%r11</code>	调用者保存	<code>%r12</code>	被调用者保存
<code>%r13</code>	被调用者保存	<code>%r14</code>	被调用者保存
<code>%r15</code>	被调用者保存		

递归过程

1. 简单例子

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

寄存器	用途	类型
%rdi	x >> 1	递归参数
%rbx	x & 1	被调用者保存

```
pcount_r:
    movl    $0, %eax
    testq   %rdi, %rdi
    je      .L6
    pushq   %rbx
    movq    %rdi, %rbx
    andl    $1, %ebx
    shrq    %rdi # (by 1)
    call    pcount_r
    addq    %rbx, %rax
    popq    %rbx
.L6:
    ret
```

递归过程是对称的，调用栈的操作呈现出加必有减、push 必有 pop 的规律。递归函数调用时，调用者将相关寄存器压栈，调用完毕后，栈中的数据会被恢复。

2. Red Zone

超出栈顶 `%rsp` 所指向位置 128 字节的区域是保留的（有时可以直接使用）。如果栈顶已在实际栈顶，则此机制无法再使用。

3. RIP 相对寻址

在链接后，所有的数据和代码会存在一起，程序中可以通过 RIP 相对寻址来访问全局变量和静态函数。通过 RIP 计算地址时，编译后的程序可以根据偏移量直接访问。

一些32位系统相关

- 编译开关：
 - `-m32`：指示编译为 32 位程序。
- 32位系统传参：

在 32 位系统中，所有参数都通过栈传递。栈从右往左压栈，子函数可以通过 `%rsp` 访问传入参数的地址。
- 32位调用示例：

32 位系统的标准调用过程如下图所示：

<pre>void swap(int *xp, int *yp) { int t0 = *xp; int t1 = *yp; *xp = t1; *yp = t0; }</pre>	<pre>swap: pushl %ebp movl %esp,%ebp pushl %ebx movl 12(%ebp),%ecx movl 8(%ebp),%edx movl (%ecx),%eax movl (%edx),%ebx movl %eax,(%edx) movl %ebx,(%ecx) movl -4(%ebp),%ebx movl %ebp,%esp popl %ebp ret</pre> 建立栈帧 函数主体 退出 leave指令
---	---

数据/代码的内存地址综合

数据、函数、指令“定位”小结

	编译	链接 (静态)	说明
局部变量(非静态)	√		存于栈内 or 寄存器 or 被优化掉
全局变量		√	各个模块(.o文件)的数据段、代码段合并后, 方能确定地址
函数参数	√		通过指定的寄存器与栈传递
函数返回值	√		%rdx + %rax
跳转指令 (直接)	√		相对于PC的offset
函数返回地址	√		存于栈顶
静态函数入口地址	√	√ (与编译优化开关有关 -Og)	相对于PC的offset
全局函数入口地址		√	各个模块(.o文件)的数据段、代码段合并后, 方能确定地址

非静态全局函数/变量是可以被多个模块引用的，而静态的不会，这是实现“定位”时需要考虑的

静态链接

完成链接后产生可执行目标文件。

调用时，系统通过加载器（loader）将执行程序的代码与数据复制到内存，执行之

链接的作用

1. **组合代码**：将多个源文件链接成一个程序，而不需要将所有代码放在一个文件中。
2. **提升效率**：

- **节省时间**：修改一个源文件后，仅需重新编译该文件并重新链接。
- **节省空间**：通用函数被集成到库中，仅加载实际使用的部分。

链接的步骤

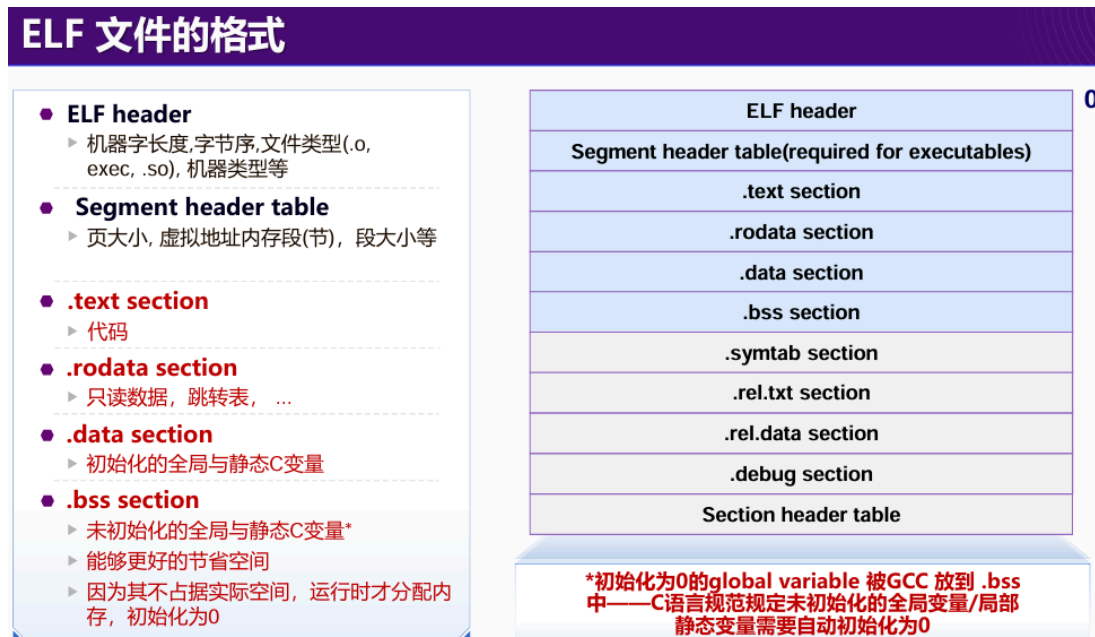
1. **符号解析**：
 - 将全局变量、静态变量和函数存入符号表中。
 - 符号表包含符号的名称、大小和位置等信息。
2. **重定位**：
 - 合并多个目标文件的数据段和代码段。
 - 将符号引用更新为具体的内存地址。

目标文件类型

1. **可重定位目标文件 (.o)**：
 - 由单一源文件生成，可与其他目标文件链接成可执行文件。
2. **可执行目标文件**：
 - 包含可直接加载到内存中执行的代码与数据。
3. **共享目标文件 (.so)**：
 - 在加载或运行时动态链接的特殊目标文件。

ELF 文件结构

目标文件的标准二进制格式之一





重点：

1. **代码段 (text)**：只读的程序指令。
2. **数据段 (data)**：已初始化的全局变量。
3. **未初始化数据段 (bss)**：未初始化的全局变量，分配为零。
4. **符号表**：存储符号定义与引用。

链接符号

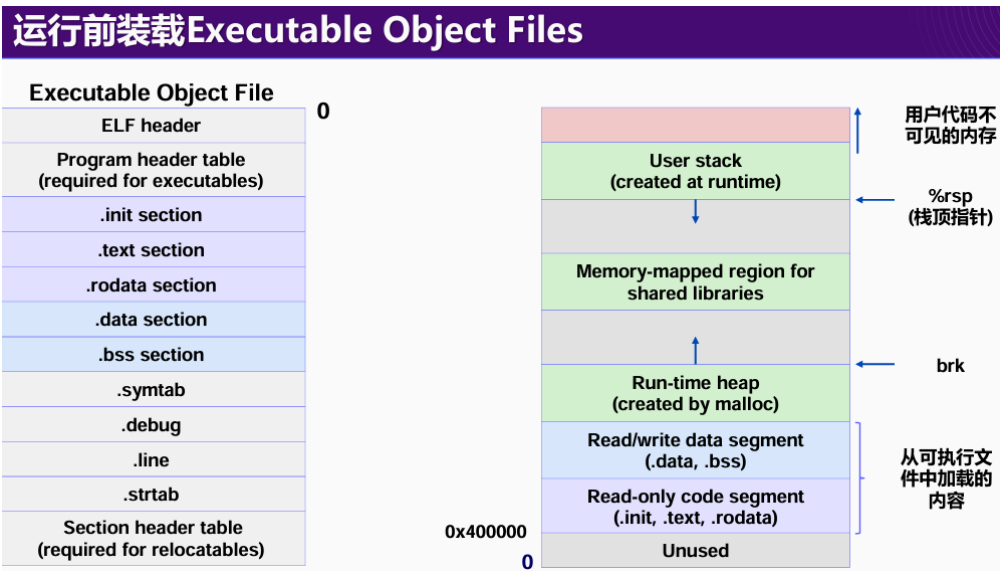
1. **全局符号**：
 - 可被其他模块引用的符号，如非静态函数或全局变量。
2. **外部符号**：
 - 当前模块中引用，但由其他模块定义的符号。
3. **局部符号**：
 - 模块内部使用的静态函数和静态变量。
 - 非静态：存储在栈或某些寄存器中 静态：存储在.bss或.data段中

重复符号解决规则

如果局部变量出现同名，编译器会在符号表中创建具有唯一名称的局部符号，例如x.1 x.2

1. 不允许多个强符号同名。
2. 强符号优先于弱符号。
3. 多个弱符号时，任选一个。

运行前装载（内存布局）



Unused 段是一段不会对应到物理地址空间的虚拟地址段，如果OS接收到这一段的地址翻译调用应该会触发异常，中止执行，以此达到防止空指针访问的效果。

常用库文件

- 1. **静态库 (archive file):**
 - 增强链接器功能，通过从库文件中提取所需函数和数据完成链接。
 - 缺点：库文件的细微变动需要所有相关执行文件进行重链接
- 2. **共享库:**
 - 在运行时动态加载，减少冗余和内存占用。
 - **位置无关代码 (PIC):**
 - 使用全局偏移表 (GOT) 间接访问全局变量（GOT表位于数据表之上）。
 - x86-64 支持使用 RIP 寄存器进行相对定位（而32位可通过 `mov (%esp), %eax` 来获得）。
 - PIC时static全局变量在汇编代码中直接使用相对于 rip（指令指针寄存器）的偏移量来访问。非static全局变量在汇编代码中是通过GOT表来定位的

缓冲区溢出及防护

缓冲区溢出

- 当写入数据超过缓冲区容量时，可能覆盖局部变量或函数返回地址。

代码注入攻击

- 输入恶意代码，覆盖关键控制数据，改变程序执行流。

防护手段

1. 避免溢出漏洞。
2. 系统级防护：
 - 栈随机化：在栈上分配随机数量的空间，使地址难以预测。
 - 栈不可执行：禁止栈中代码执行。
3. 金丝雀 (Canary)：
 - 在缓冲区上方放置特殊值，检测是否被修改。

返回导向编程 (ROP)

- 攻击方式：
 - 利用共享库等现有代码片段 (gadgets)，通过其 ret 指令执行下一片段。
 - 可以绕过栈随机化，但无法绕过金丝雀保护。
-

虚存 (Virtual Memory)

虚存是一种内存管理技术，为进程提供一个连续的虚拟地址空间，简化内存管理，并允许程序使用比物理内存更大的地址空间。

核心思想

将虚拟地址 (VA) 映射到物理地址 (PA)，通过页表管理这种映射关系。

虚存的主要功能

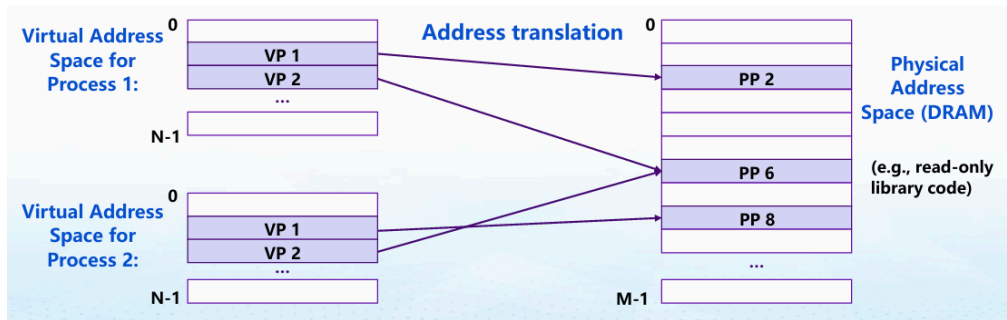
1. 使用物理内存作为虚拟内存空间的缓存

- **基础机制**：虚拟内存利用**局部性原理**，将活跃数据从硬盘等二级存储加载到物理内存，通过以下机制调度：
 - **Page Hit**：所需数据已在物理内存中，直接访问。
 - **Page Fault**：所需数据不在内存，需从硬盘加载，并可能替换一个页面。
- **抖动效应 (Thrashing)**：当工作集大小超出物理内存，频繁页面交换导致性能崩溃。

2. 简化内存管理

虚存提供统一的线性地址空间，通过页表实现地址映射。

- **页表管理映射**：虚拟地址通过页号 (VPN) 在页表中查找对应的物理页号 (PPN) 。



3. 地址空间隔离

- **进程隔离**：不同进程间地址空间相互独立。
- **内核保护**：用户程序无法访问操作系统内核数据。

常用符号

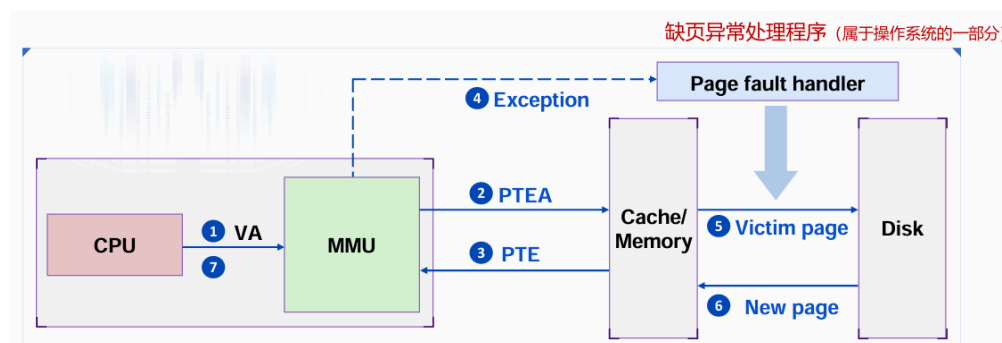
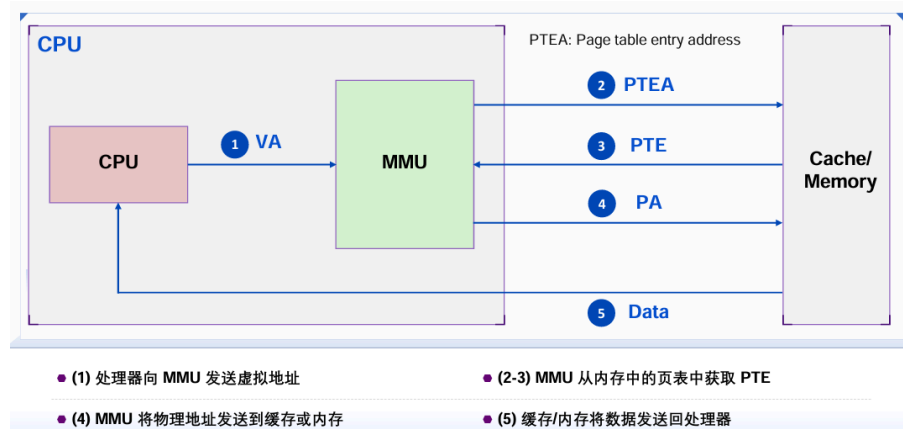
- **虚拟地址 (VA)**
 - **VPN**：Virtual page number 虚拟页号
 - **VPO**：Virtual page offset 页内偏移量，与PPO相同
- **物理地址 (PA)**
 - **PPN**：物理页号

- **PPO**：页内偏移量，与VPO相同
- **页表项 (PTE)**：记录页的状态及映射信息，如有效位、权限位等。
 - 有效位为0：触发Page Fault。
 - 无效访问权限：触发SIGSEGV（段错误）。

地址翻译流程

1. 根据VPN从页表查找对应的PPN。
2. 如果页表有效位为1，则与VPO组合生成物理地址PA。
3. 如果有效位为0，触发**异常控制流**，加载缺页，并选择替换页。

实际处理过程示意图：



TLB (Translation Lookaside Buffer)

作用：TLB是MMU中的硬件缓存，用于加速页表查询，存储常用页表项。

考虑TLB的查找过程：

1. 先把VPN看成**TLBI**（索引）和**TLBT**（标签）在TLB表里找。
- 实战经验：假设快表的组织形式为n-way，最多存储m条目，即m/n条每组，则

$2^{TLBI} = m/n, VPN - TLBI = TLBT$ 。其中右边（低位）开始找TLBI

2. **TLB命中**：拼接PPN和VPO，生成PA。
3. **TLB Miss**：访问页表查找PPN，可能触发**Page Fault**。

实战示例

页表

Only show first 16 entries (out of 256)

VPN	PPN	Valid
00	28	1
01	-	0
02	32	1
03	02	1
04	-	0
05	16	1
06	-	0
07	-	0
08	13	1
09	17	1
0A	09	1
0B	-	0
0C	-	0
0D	2D	1
0E	11	1
0F	0D	1

TLB

- 16个页表项
- 4-way set associative (四路组相联，一个set中的4路均可以存储某一个entry)

Diagram illustrating the TLB structure and address mapping for Virtual Address 0x03D4.

Virtual Address: 0x03D4

TLB Index (TLBI) = 4 (0100). TLB Set = 0 (0000). TLB Entry 0: Tag = 03, PPN = 2D, Valid = 1. TLB Hit? Y. Page Fault? N. PPN = 0x0D.

Physical Address

PPN = 0x0D, PPO = 0x00. Physical Address = 0x0D00.

Virtual Address: 0x0B8F

TLB Index (TLBI) = 3 (0011). TLB Set = 1 (0001). TLB Entry 1: Tag = 03, PPN = 2D, Valid = 1. TLB Hit? N. Page Fault? Y. PPN = TBD.

Physical Address

PPN = TBD, PPO = 0x00. Physical Address = TBD000.

Virtual Address: 0x0020

TLB Index (TLBI) = 0 (0000). TLB Set = 0 (0000). TLB Entry 0: Tag = 00, PPN = 00, Valid = 0. TLB Hit? N. Page Fault? N. PPN = 0x28.

Physical Address

PPN = 0x28, PPO = 0x00. Physical Address = 0x2800.

内存映射 (Memory Mapping)

通过关联硬盘上存储的各类对象来初始化虚存各个区域的过程

对象类型：

1. **普通文件页**：从硬盘加载程序文件中的代码段、数据段等。
2. **匿名页**：初始为全零的页面，写入（dirty）后会像普通页面一样存储。
dirty页在内存和一个特殊的硬盘交换文件（swap file）之间来回复制

当初始化全局数据被改后，不应该存回rodata，故只能存在swap file中

页面共享与写时复制：

- **共享页面**：只读资源（如动态库）。
- **写时复制**：修改共享页面时，系统为进程创建私有副本。

一些理解

缓存

分为指令缓存和数据缓存。

缓存是以块（通常为 64 字节或更多）为单位加载的，这意味着相邻的数据也被一并加载。故一般连续的指令或数据只需缓存一次就可以一直命中（局部性）

为什么不是通过这个地址加上一定的值前往下一个地址呢？

在虚拟内存系统中，内存是以页面为单位管理的。如果访问的数据跨越了不同的页，就需要通过页表来找到数据的实际物理位置。简单的地址加法只能在同一页内有效，而当访问跨页时，必须根据页表项重新计算物理地址。

动态内存分配

动态分配器（如malloc）通过管理虚拟内存堆区域，实现内存分配。

常见约束：

- 即时响应分配请求。
- 空闲块必须满足对齐要求。
- 已分配块不可移动。

分配策略

1. 隐式链表

- 每个块包含一个header，记录块大小及状态（分配/空闲）。

2. 搜索方式

- **First Fit**: 从头找到第一个足够大的空闲块。
- **Next Fit**: 从上次搜索结束处继续查找（较差）。
- **Best Fit**: 找到最小但足够大的空闲块。

3. 块合并

- **向后合并**: 当前块与后方空闲块合并。
- **双向合并**: 与前后空闲块合并，但需记录前块状态。

性能指标

- **吞吐率**: 单位时间内完成的分配请求数。
- **峰值利用率**: 最大有效负载与堆大小的比值。
- **碎片化**:
 - **内部碎片化**: 块大小大于有效负载。
 - **外部碎片化**: 堆内有足够空闲内存但分布不连续。

异常处理

till now, 存在两种改变控制流的机制: 一种是跳转 (包括无条件跳转 `jump` 和有条件跳转 `branch`) , 另一种是过程调用与返回 (`call/return`) 。

这些机制都是响应程序内部的执行状态变化。而异常处理则不同, 它是将控制权转移到操作系统, 以响应某些特定的事件。异常处理通过异常处理向量进行, 该向量会依次按类型匹配异常。

异步异常 (中断)

- **定义**: 异步异常是由“外部事件”引起的, 通常是外部设备触发的。这类异常也被称为中断。
- **例子**: 例如I/O中断, 当外部设备需要CPU处理某些事务时, 它会触发一个中断。

同步异常

同步异常是由正在执行的指令引起的, 可以分为以下三类:

1. 陷阱 (Traps)

- **描述**: 程序“故意”引发的异常，用于实现特定的功能。
- **例子**: 系统调用、程序断点、特殊指令等。

2. 故障 (Faults)

- **描述**: 程序“无意”引发的异常，通常是可恢复的错误。
- **例子**: 缺页错误、非法指令、地址越界、算术溢出等。

3. 终止 (Aborts)

- **描述**: 程序“无法”恢复的错误，通常会导致程序终止。
- **例子**: 如操作系统的蓝屏错误，表示系统遇到了严重错误，无法继续执行。

进程

定义: 进程是程序的一个运行实例。

二要素: 内存 (Mem) + CPU。

多进程工作流程

1. 保存当前所有寄存器内容到内存中。
2. 调度下一个进程运行。
3. 加载保存的寄存器并切换地址空间（上下文切换）。
 - 上下文切换由操作系统内核 (kernel) 中的一段代码管理。
 - 该代码作为进程的一部分运行在虚拟内存空间的高位地址。

并发进程

定义: 两个进程的执行在时间上重叠。

fork

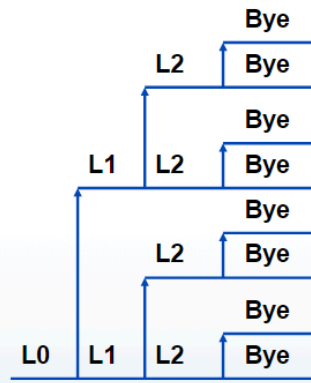
功能: 创建与当前进程相同的子进程。

- **返回值**: 子进程返回 0；父进程返回子进程的进程号 (pid)。
- **调用顺序**: 父进程和子进程的执行顺序不确定。

fork 细节

- **原理：** fork 是把当前进程接下来的操作复制一份。

```
void fork3()
{
    printf("L0\n");
    fork();
    printf("L1\n");
    fork();
    printf("L2\n");
    fork();
    printf("Bye\n");
}
```



- **虚拟地址：**
 - i. 创建新进程时，虚拟地址空间会包含当前 `mm_struct`、`vm_area_struct` 和页表的精确副本。
 - ii. 所有页面标记为只读；`vm_area_struct` 标记为写时复制（Copy-On-Write, COW）。

常见考题： 判断父子进程的合理输出。

- **解题方法：** 画图分析。
- **注意：** 涉及父子进程的问题要特别关注写时复制的机制。

exit

功能： 退出进程，通常以状态 `0` 表示成功。

- **区别：**
 - `return`：返回函数调用处。
 - `exit(0)`：等价于主程序的 `return`。

atexit

功能： 注册退出时要执行的函数，按注册顺序的逆序执行。

示例：

```

void cleanup() {}
void fork6() {
    atexit(cleanup);
    atexit(cleanup2);
    atexit(cleanup3);
    // 退出时，依次执行 cleanup3 -> cleanup2 -> cleanup
}

```

pthread

1. 线程创建和终止

- `pthread_create`

创建新线程，并指定执行的函数。create后即加入调度池。

```

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
    void *(*start_routine)(void *), void *arg);

```

- `thread`：用于存储线程标识符（ID）。
- `attr`：指定线程属性，可设为 `NULL` 使用默认属性。
- `start_routine`：线程执行的函数，函数需返回 `void*` 并接收 `void*` 参数。
- `arg`：传递给 `start_routine` 的参数。

- `pthread_exit`

终止线程执行并返回一个值给 `pthread_join`。

```

void pthread_exit(void *retval);

```

2. 线程同步

- `pthread_join`

等待一个线程结束，获取其退出状态。

```

int pthread_join(pthread_t thread, void **retval);

```

- `thread`：目标线程的 ID。
- `retval`：用于存储线程退出时返回的值。

僵尸进程 (zombies)

定义：进程终止后仍会消耗系统资源的进程。

回收机制 (Reaping)

- **原理：**
操作系统内核将子进程的退出状态信息传递给父进程后，释放子进程的剩余资源。
- **特殊情况：**
 - 如果父进程未回收子进程资源并中止，子进程将由 `init` 进程接管。

回收方法

1. `wait()` :
 - 阻塞直到任一子进程终止，返回子进程的进程号。
 - 若传入指针不为空，指针会被设置为子进程终止原因的状态。

```
int wait(int *status);
```

2. `waitpid()` :
 - 阻塞直到指定的子进程终止，返回其进程号。

```
int waitpid(int pid, int *status, int options);
```

execve

功能：在当前进程中加载并运行新程序。

- **特点：**不会返回（除非出现错误）。
- **覆盖内容：**当前进程的代码、数据和堆栈。
- **保留内容：**进程号、打开的文件描述符和信号上下文保持不变。

» 在当前进程中加载并运行一个新程序

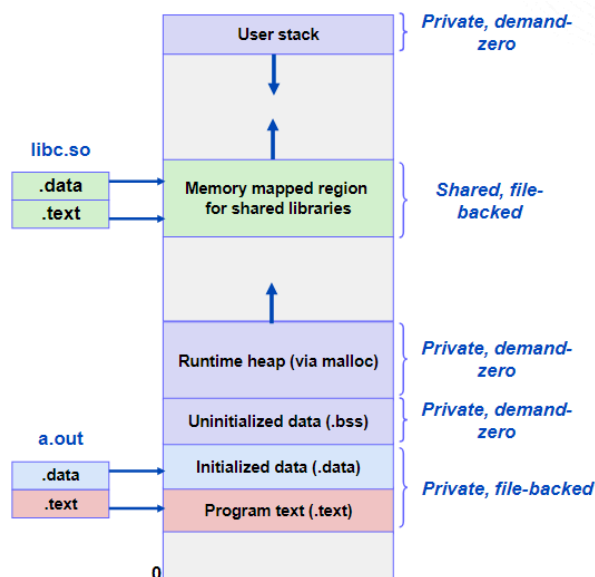
» 释放旧区域的vm_area_struct和页表

» 为新区域创建vm_area_struct's和页表

- 正文段与初始化数据来自于相应的可执行目标文件

» 将PC设置为正文段的入口点

- 回忆“虚存”一讲：按需分配物理页面（由“页缺失”触发）



信号处理概述

信号处理

壳程序 (Shell)

- **定义：**壳程序是代表用户运行程序的应用程序，例如 `sh`、`csh`、`bash` 等，它们负责执行一系列“命令读取/执行”的步骤。

后台任务

- **特点：**允许用户在不等待进程结束的情况下继续操作。
- **示例：**

```
(sleep 7200; rm /tmp/junk) & # 设置为后台任务
```

异常控制流 (ECF)

- **问题：**Shell 不知道后台作业的结束时间，无法及时回收 (reap) 退出的后台进程。
- **解决方案：**通过内核发送信号的机制通知 Shell，提醒后台任务的完成情况。

信号概述

定义

- **信号**：内核发送给进程的小消息，用于通知某些系统事件的发生。
- **特点**：由唯一的整数 ID 标识，信号的传递信息仅包括信号 ID 和信号到达的事实。

常见信号及默认动作

- **SIGINT (2)**：中断信号，通常由 `Ctrl+C` 触发，默认终止进程。
- **SIGKILL (9)**：强制终止进程，无法被捕获或忽略。
- **SIGSEGV (11)**：段错误信号，因访问非法内存触发，默认行为是终止并生成核心转储。
- **SIGALRM (14)**：定时器信号，默认终止进程。
- **SIGCHLD (17)**：子进程停止或终止信号，默认忽略。

信号的发送与接收

1. 信号的发送

- **来源**：
 - i. 内核检测到异常事件（如非法内存访问触发 `SIGSEGV`）。
 - ii. 使用 `kill` 系统调用发送信号（如 `kill -9 PID`）。
- **键盘触发**：
 - `Ctrl+C`：发送 `SIGINT`，终止前台进程。
 - `Ctrl+Z`：发送 `SIGTSTP`，暂停前台进程。

2. 信号的接收

- **接收时机**：当进程被调度恢复执行时处理信号。
- **信号状态**：
 - i. **挂起 (pending)**：信号已到达，但未被接收。
 - ii. **阻塞 (blocked)**：信号被进程屏蔽，需解除阻塞后接收。

3. 信号的处理

- **处理方式**：

- i. 忽略信号。
 - ii. 终止进程（可选是否生成核心转储）。
 - iii. 捕获信号，通过信号处理程序（signal handler）进行处理。
- **优先级**：挂起信号按 ID 从低到高依次处理。

信号处理程序（Signal Handlers）

1. 设置信号处理程序

- **函数原型**：

```
signal(int signum, handler_t *handler);
```

- **signum**：信号编号。
- **handler**：信号处理程序，可为以下之一：
 - **SIG_IGN**：忽略信号。
 - **SIG_DFL**：恢复默认处理。
 - 自定义函数：实现自定义信号处理逻辑。

2. 信号处理程序的特点

- **与进程并发执行**：在主执行流外运行，但共享进程的内存空间。
- **独立的栈和上下文**：处理程序拥有自己的栈和上下文，但可访问和修改全局变量。

3. 示例代码

```
void int_handler(int sig) {
    printf("Process %d received signal %d\n", getpid(), sig);
    exit(0);
}

void fork_example() {
    signal(SIGINT, int_handler); // 设置信号处理程序
    for (int i = 0; i < 3; i++) {
        if (fork() == 0) {
            while(1); // 子进程进入无限循环
        }
    }
}
```

有意思的例子

```
int ccount = 0;
void child_handler(int sig)
{
    int child_status;
    pid_t pid = wait(&child_status);
    ccount--;
    safe_printf(
        "Received signal %d from process %d\n",
        sig, pid);
}
void fork14()
{
    pid_t pid[N]; //N > 2
    int i, child_status;
    ccount = N;
    signal(SIGCHLD, child_handler);
    for (i = 0; i < N; i++)
        if ((pid[i] = fork()) == 0)
            sleep(1); /* describe child */
    while (ccount > 0)
        pause(); /* Suspend until signal occurs */
}
//signal1.c
```

被挂起的信号是没有队列的

- 对于每种信号类型，只有一个比特表示该信号是否挂起
- 即便是多个进程发送了这个信号

因信号无法缓存，多个同类信号同时到达的话会丢失若干

```
linux> ./forks 14
Received SIGCHLD signal 17 for process 21344
Received SIGCHLD signal 17 for process 21345
```

错因：同类型信号处理前只会留下一个
标准收割写法：

》 必须检查所有的终止子进程

- 通常是 `wait ()` 的循环

```
void child_handler2(int sig)
{
    int child_status;
    pid_t pid;
    while ((pid = waitpid(-1, &child_status, WNOHANG)) > 0) {
        ccount--;
        safe_printf("Received signal %d from process %d\n", sig, pid);
    }
}

void fork15()
{
    ...
    signal(SIGCHLD, child_handler2);
    ...
} //signal2.c
```

```
greatwhite> forks 15
Received signal 17 from process 27476
Received signal 17 from process 27477
Received signal 17 from process 27478
Received signal 17 from process 27479
Received signal 17 from process 27480
greatwhite>
```

WNOHANG=不要挂起：主任务还要跑

异步信号安全

定义

- **异步信号安全**：信号处理程序仅调用不会被信号中断的函数。
- **问题**：若调用非安全函数（如 `printf`），可能导致缓冲区破坏、死锁等问题。

示例

- 替换 `printf` 为异步信号安全的 `safe_printf`。

非本地跳转： `setjmp` 和 `longjmp`

概述

- 提供从深层栈跳转至浅层栈的机制，用于异常情况的处理。

函数说明

1. `int setjmp(jmp_buf env)`
 - 保存当前环境信息，用于后续跳转。
 - **返回值**：直接调用返回 `0`；通过 `longjmp` 返回其参数值。
2. `void longjmp(jmp_buf env, int val)`

- 恢复 `setjmp` 保存的环境，实现跳转。
- 跳转后从 `setjmp` 返回，不执行 `longjmp` 之后的代码。

局限性

1. **跳转限制：**
 - 不能实现跨函数调用的跳转，仅能在调用链内跳转。
 2. **代码风险：**
 - 可能跳过局部变量清理，导致资源泄漏或状态异常。
 3. 性能开销大
-

IO处理

Unix I/O

基本概念

- **Unix文件：**本质是一个字节序列，IO设备（如硬盘、终端）也被视为文件。
- **文件类型：**常规文件、目录文件等。
- **文件尾部：**文件尾-1是内容的实际尾部位置。

基本操作

1. **打开与关闭文件**
 - `open()`：打开文件。
 - `close()`：关闭文件。
 - 提示：
 - a. 进程终止时，内核会自动关闭所有未关闭的文件描述符。
 - b. 数据写入磁盘是延迟的，需调用 `fsync()` 确保数据同步。
 - c. 始终检查 `close()` 的返回值，避免关闭失败导致的潜在错误。
2. **读写文件**

- `ssize_t read(int fd, void *buf, size_t count);`：从文件中读取内容到内存，同时更新文件位置。
buf：指向存储读取内容的缓冲区的指针。
count：希望读取的字节数
 - 返回值：读取的字节数，小于请求字节数可能是文件尾或终端输入。
- `ssize_t write(int fd, const void *buf, size_t count);`：将内存数据写入文件，同时更新文件位置。
 - 返回值：写入的字节数。

3. 改变文件位置

- `lseek()`：调整文件的读写位置。

4. 标准文件描述符

- 标准输入 (stdin)：文件描述符 0
- 标准输出 (stdout)：文件描述符 1
- 标准错误 (stderr)：文件描述符 2

文件元数据

- **定义**：由内核维护的文件信息。
- **访问方式**：
 - `stat()`：通过文件名访问。
 - `fstat()`：通过文件描述符访问。

元数据结构

```
struct stat {
    dev_t st_dev;           // 设备号
    ino_t st_ino;           // inode号
    mode_t st_mode;         // 文件权限和类型
    nlink_t st_nlink;       // 硬链接数
    uid_t st_uid;           // 所有者用户ID
    gid_t st_gid;           // 所有者组ID
    off_t st_size;          // 文件大小（字节数）
    time_t st_atime;        // 上次访问时间
    time_t st_mtime;        // 上次修改时间
    time_t st_ctime;        // 状态更改时间
};
```

访问文件目录

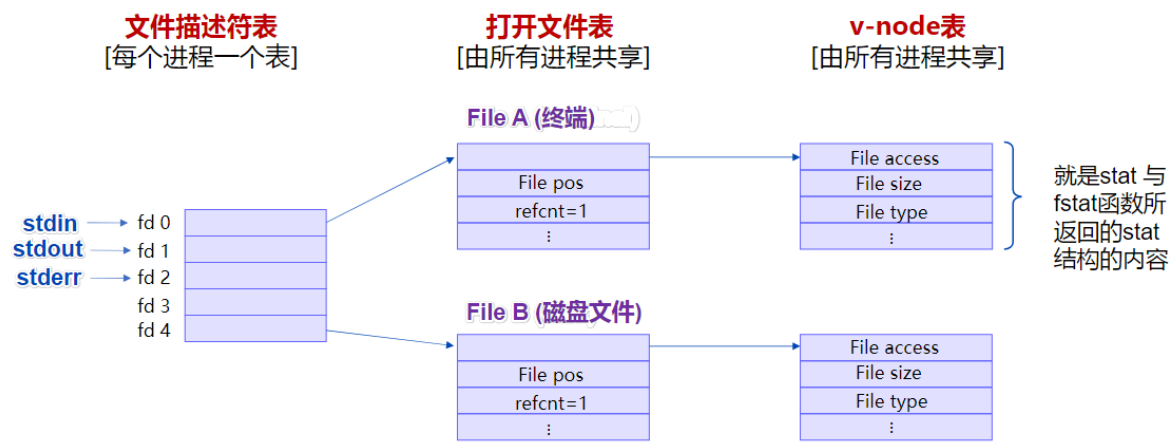
目录结构 dirent

```
struct dirent {
    long d_ino;           // 索引节点号
    off_t d_off;          // 在目录文件中的偏移
    unsigned short d_reclen; // 文件名长度
    unsigned char d_type;  // 文件类型
    char d_name[NAME_MAX+1]; // 文件名
};
```

文件共享

0. 基本情况

两个描述符表示两个不同的打开的文件：描述符1 (stdout)指向终端，描述符4指向打开磁盘文件



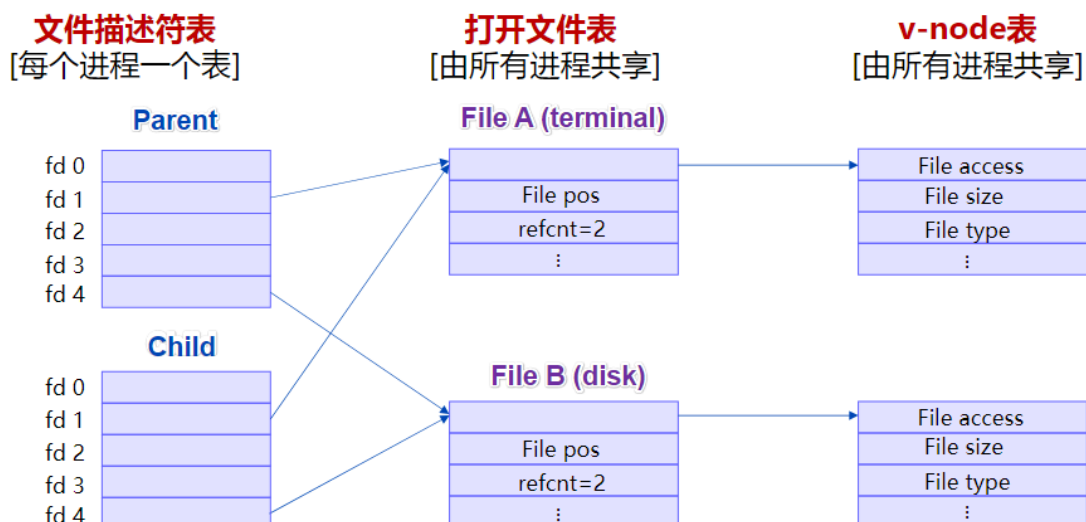
其中 refcnt 是引用计数。各文件有各自的读写位置

1. 共享方式1

例：使用相同的文件名调用open两次
表现：上图的FileA和FileB指向v-node表中同一项
(是新加表项而非refcnt++;)

2. 共享方式2

示例：子进程继承了其父进程打开的文件
调用fork()之后：子进程的文件描述符表与父表相同，而打开文件表的相应计数
refcnt++



此时父进程和子进程的读写会相互影响
(可以使用fcntl(fd, F_SETFD, FD_CLOEXEC)来改变这一默认行为)

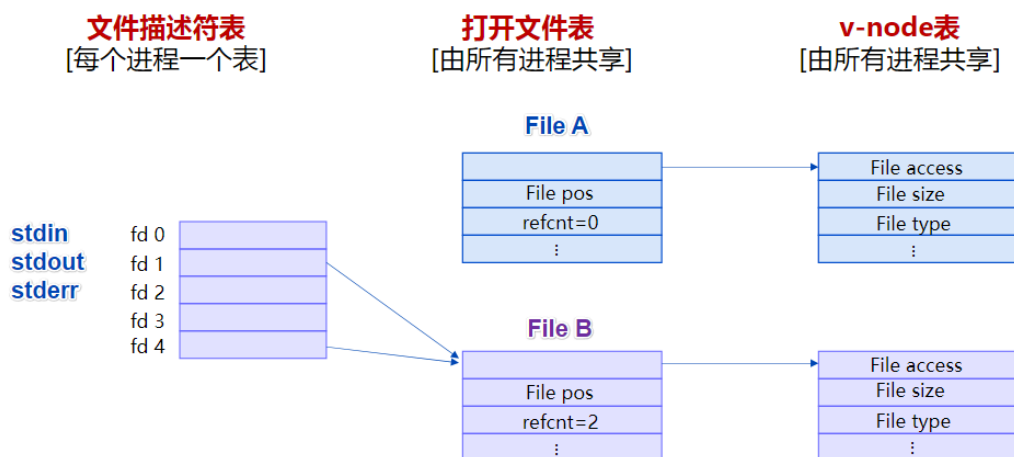
访问重定向

例：壳程序的重定向 `unix > ls > foo.txt`

实现：调用dup2(oldfd, newfd) 将（每进程）文件描述符表项从oldfd复制到表项newfd
包括文件的偏移量、访问模式（如只读、只写、读写）、以及状态标志（如是否为追加模式）都会被覆盖

第二步：调用dup2 (4,1)

使得fd=1 (stdout)指向fd=4所指向的磁盘文件



例子：

```
#include "csapp.h"
int main(int argc, char *argv[]){
    int fd1, fd2, fd3; //噢，为什么变量类型是int呢？
    char *fname = argv[1];
    fd1 = Open(fname, O_CREAT|O_TRUNC|O_RDWR, S_IRUSR|S_IWUSR); /*如果无就创建，如果有就清空文件内容*/
    Write(fd1, "pqrs", 4);
    fd3 = Open(fname, O_APPEND|O_WRONLY, 0); /*每次写入都是追加到文件尾，lseek对它不起作用*/
    Write(fd3, "jklmn", 5);
    fd2 = dup(fd1); /* 复制一份自己给f2 refcnt++ */
    Write(fd2, "wxyz", 4);
    Write(fd3, "ef", 2);
    return 0; /*pqrsxyznejf*/
}
```

标准 I/O

C 标准库（`libc.so`）提供了一系列高级 I/O 函数：

- 文件打开/关闭：`fopen()`、`fclose()`
- 文件读写：`fread()`、`fwrite()`
- 文本读写：`fgets()`、`fputs()`
- 格式化读写：`fscanf()`、`fprintf()`

数据流与缓存

- **数据流**：`FILE*` 是文件描述符与缓存的组合。
- **内置数据流**：程序启动时自动打开 `stdin`、`stdout` 和 `stderr`。
- **缓存机制**：写入 `\n` 或调用 `fflush()` 时刷入文件。

Unix I/O 与标准 I/O 的比较

特性	Unix I/O	标准 I/O
优点	通用高效	缓存提高效率
	支持元数据访问	自动处理不足值
缺点	处理不足值较复杂	不支持信号处理

特性	Unix I/O	标准 I/O
	缓存操作需手动	无法访问元数据

选择 I/O 函数

- **标准 I/O**：处理磁盘和终端文件时使用。
- **UNIX I/O**：需要信号安全或高性能时使用。
- **RIO 包**：处理网络套接字时使用。

特殊情况：处理二进制文件

- **不能使用的函数**：
 - 基于行处理的 I/O： `fgets` , `scanf` , `printf` , `rio_readlineb` 。
 - 替代方案：使用 `rio_readn` 或 `rio_readnb` 。
- **不能直接操作的字符串函数**：
 - 例如 `strlen` 、 `strcpy` , 会将字节 `0` 解释为字符串结尾。

线程与线程同步基础

我们眼中的进程

1. **定义**
进程是操作系统中的基本执行单元，其本质为 **进程上下文 + 代码、数据、堆栈**。
2. **进程上下文**
 - **处理器上下文 (CPU context)**：包含通用寄存器、条件码、栈顶指针 (SP) 和程序计数器 (PC)，用于保存 CPU 执行状态。
 - **内核上下文 (Kernel context)**：包含虚拟内存相关结构（如页表、 `task & mm structs` 、 `vm_area_struct` ）、文件描述符表、堆指针 (brk pointer) 和信号上下文，用于进程管理和系统资源协调。
3. **多线程视角**
从另一个视角来看， **进程 = 线程 + 代码、数据、内核上下文**。线程是执行的基本单元，进程是线程的执行环境。
4. **线程与进程的关系**

- **线程的处理器上下文 (CPU context)**：每个线程独立维护通用寄存器、条件码、栈顶指针 (SP) 和程序计数器 (PC)。每个线程有自己的线程编号TID.
- **进程的共享资源**：所有线程共享同一进程的代码段、数据段和内核上下文，包括共享库、运行时堆、读/写数据和只读代码/数据。
- 线程的资源开销比进程小，易于在线程间共享数据。但是!无意的共享可能会导致微妙的、难以重现的错误!

5. 一些理解

进程是程序的运行实例，线程是进程中的执行流共享进程的地址空间和资源。协程是轻量级的线程由用户态调度不依赖操作系统内核
线程加速靠并行+共享内存，切换成本更小但需协调数据同步

共享

定义: 一个变量x是共享的，当且仅当多个线程引用x的某个实例

概念模型

1. 多个线程在单个进程内运行
 - 每个线程有独立的线程上下文（包括线程号、栈、栈指针、PC、条件码以及通用寄存器）
2. 所有线程共享其余的进程资源
 - 进程虚拟空间的代码、数据、堆、共享库
 - 打开的文件以及设置好的信号处理函数 (handler)

操作层面

1. 寄存器值是严格隔离和受到保护的，但虚存并不如此
2. 任何线程可以读、写任何其他线程的栈

变量实例在内存中的映射

1. **全局变量**：定义在所有函数外面的变量
虚拟内存包含全局变量的唯一实例
2. **局部变量**：定义在函数内部的变量，无static属性
每个线程栈包含每个局部变量的一个实例
3. **局部静态变量**：定义在函数内部，有static属性
虚拟内存包含局部静态变量的唯一实例

线程间共享vs编译中的共享：

特性	线程间共享	编译链接的共享
发生阶段	运行时	编译时或加载时
共享范围	同一进程内的线程	多个进程之间
共享内容	进程内存（代码段、数据段、堆）	静态或动态库的代码段
共享方式	自动共享（基于地址空间）	静态链接：代码拷贝 动态链接：共享内存
同步需求	需要同步机制防止数据竞争	动态链接库的代码段是只读的，无需同步
性能开销	无需额外拷贝，但加锁有一定开销	静态链接：无运行时开销 动态链接：有加载和地址映射开销
隔离性	地址空间共享，错误可能影响整个进程	动态库的代码段隔离性强，不影响进程数据
适用场景	并发任务间协作，访问全局数据	代码复用，减少内存占用

判断是否是线程间共享的就看它是否被多个线程引用

互斥

临界区的执行不能被打断，确保一个线程在访问临界区时，其它线程不能进入该临界区。

问题示例

```
movq cnt(%rip), %rax    # 读取cnt的值到寄存器
addq $1, %rax           # 计数值加1
movq %rax, cnt(%rip)    # 写回cnt
```

- 如果线程二在线程一完成整个指令序列之前访问 `cnt`，可能会读取未更新的值 `cnt`，导致最终结果不确定。
- 问题本质：临界区的执行没有互斥，线程之间并发导致数据竞争。

1. 改用复杂指令：

```
incq cnt(%rip)    # 将 cnt 加1
```

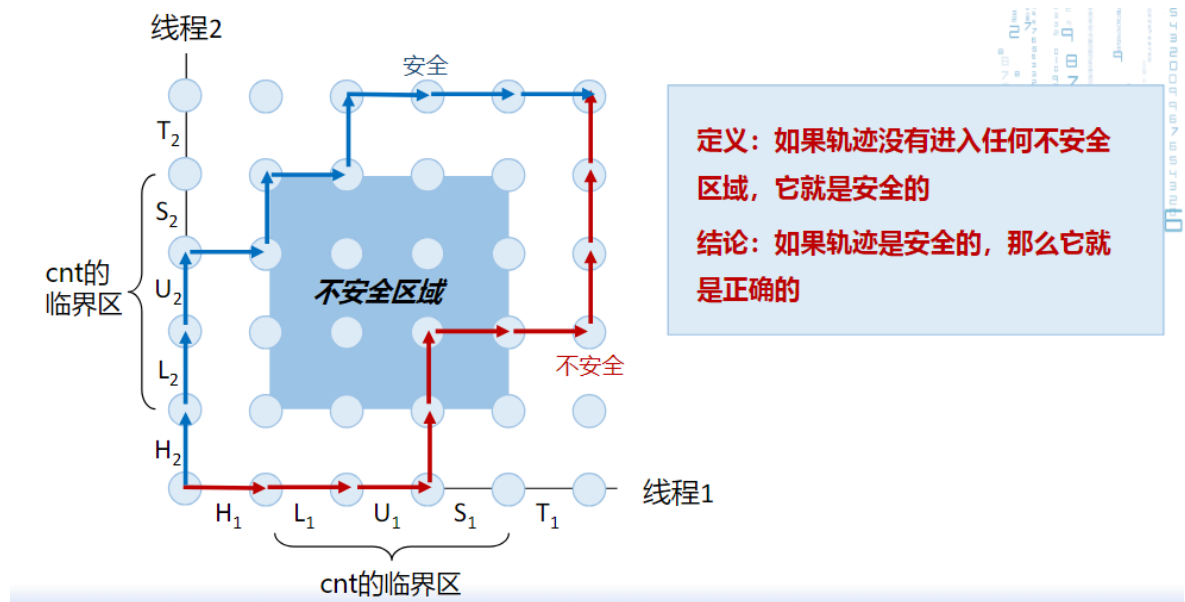
- **复杂指令集 (CISC) 的拆分问题**：复杂指令可能被拆解成多个微指令在 CPU 内部执行，因此依旧可能引发竞争。
- **总线锁定问题**：如果需要真正的原子性，可能需要锁定总线或特定的缓存行（使用如 `LOCK` 前缀的指令），这种机制会显著降低性能。

2. 单核情况：

- 单核 CPU 上虽然线程切换只能交替进行，但并不能保证每次调度点都避开临界区执行，因此竞争依然可能发生，只是概率较低。

确保安全的执行轨迹

- 用进度图表示可能的执行状态



临界区的边界可以走，但不能踏入

为了确保线程对临界区的互斥访问，需要同步线程的执行。

- **解决方案：使用同步机制**：最经典的机制是 **信号量 (Semaphore)** 或 **互斥锁 (Mutex)**。

信号量

定义

信号量是一个变量，表示共享资源的可用状态，并支持以下两种原子操作：

1. P(S) 操作 (wait 或 down)：

- 如果 $S > 0$ ，则 S 减 1，线程继续执行。

- 如果 $S = 0$ ，线程阻塞，直到 $S > 0$ 。
2. **V(S) 操作 (signal 或 up) :**
 - 将 S 加 1，并唤醒阻塞的线程（如果有）。

操作系统内核保证信号量的更新是**原子的**，即不会被其它线程中断。

实际应用：保护临界区

1. **关联信号量:**
 - 每个共享变量或一组相关共享变量，关联一个唯一信号量（如 `mutex`），初始值为 1。
 - 临界区用 `P(mutex)` 和 `V(mutex)` 包裹，确保互斥访问。
2. **信号量类型:**
 - **二值信号量 (Binary Semaphore)**：值始终为 0 或 1，用于互斥。
 - **计数信号量 (Counting Semaphore)**：用作资源计数器，表示某种资源的剩余数量。
3. **效率问题:**
 - 频繁对信号量进行操作会导致**全局变量的持续内存访问**，引发性能瓶颈。
 - 因此应尽量缩小临界区范围，减少锁的持有时间。

例子：生产者/消费者问题

- **生产者**：生产数据，将数据放入缓冲区，通知消费者。
- **消费者**：消费数据，从缓冲区中取出数据，通知生产者。
- **限制条件:**
 - 缓冲区有固定大小。
 - 生产者需要等待空闲缓冲槽，消费者需要等待数据可用。

信号量实现

1. **所需信号量:**
 - `mutex`：保护缓冲区的互斥访问。
 - `slots`：计数缓冲区中可用的槽位，初始值为缓冲区大小 N 。
 - `items`：计数缓冲区中可用的数据条目，初始值为 0。
2. **代码实现:**

```
// 生产者
P(slots);           // 等待有空闲槽位
P(mutex);           // 获取缓冲区互斥访问权
insert_item();       // 插入数据到缓冲区
V(mutex);           // 释放互斥锁
V(items);           // 通知消费者有新数据

// 消费者
P(items);           // 等待有数据
P(mutex);           // 获取缓冲区互斥访问权
remove_item();       // 从缓冲区移除数据
V(mutex);           // 释放互斥锁
V(slots);           // 通知生产者有空闲槽位
```

其中获取锁的顺序很重要，而释放的顺序不重要

竞争条件

当线程同时访问和修改共享变量，且没有正确同步时，可能发生未定义行为。导致结果依赖于线程调度的随机性。

死锁

定义：

死锁是指两个或多个线程互相等待对方释放资源，导致所有线程都无法继续执行。

避免死锁的策略：

1. **资源分配有序性**：按照统一顺序请求资源，避免循环等待。
2. **尝试获取**：设置超时机制或尝试获取资源失败时退出。
3. **资源剥夺**：允许某些资源被中断并重新分配。